

Reviewer Pack — Noncommutative L^∞ Norm Lower Bounds Disjointness Communicat...

Entry a956dba16ef5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 17:13:02 UTC

Noncommutative L^∞ Norm Lower Bounds Disjointness Communication Complexity

Entry ID: a956dba16ef5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 17:13:02 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For the DISJOINTNESS matrix $M \in \{0,1\}^{\{n \times n\}}$, the noncommutative L^∞ norm $\|M\|_{\infty} \geq c\sqrt{n}$ for some absolute constant $c > 0$. This norm is defined as the maximum singular value of the operator M acting on the space of matrices with trace-zero constraints.

Rationale (proposer's reasoning):

Noncommutative L^p norms capture operator-level structure of communication matrices, while DISJOINTNESS's $\Omega(\sqrt{n})$ lower bound suggests deep algebraic constraints. The trace-zero constraint enforces nontrivial matrix-valued analysis, linking operator norms to communication complexity.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f73c89c7c2cec841

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.9s

5.1 Generated Python source

```
import random
import math

def generate_disjointness_matrix(n):
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                M[i][j] = 1
                M[j][i] = 1
    return M

def max_singular_value(M):
    m, n = len(M), len(M[0])
    U = [[0] * n for _ in range(n)]
    S = [0] * min(m, n)
    V = [[0] * m for _ in range(m)]

    # Center the matrix
    M_centered = [[M[i][j] - (sum(M[i]) + sum(M[j])) / (2 * n) for j in range(n)] for i in range(m)]

    # Compute SVD using power iteration method
    v = [1.0] * n
    for _ in range(100):
        u = [sum(M_centered[i][j] * v[j] for j in range(n)) for i in range(m)]
        norm_u = math.sqrt(sum(x**2 for x in u))
        u = [x / norm_u for x in u]
        s = sum(u[i] * M_centered[i][j] * v[j] for i in range(m) for j in range(n))
        v = [sum(M_centered[i][j] * u[i] for i in range(m)) for j in range(n)]
        norm_v = math.sqrt(sum(x**2 for x in v))
        v = [x / norm_v for x in v]

    S[0] = s
    U[0] = u
    V[0] = v

    return max(S)
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    M = generate_disjointness_matrix(n)
    norm = max_singular_value(M)
    c = 0.5
    conjecture_holds = norm >= c * math.sqrt(n)
    counterexample = "" if conjecture_holds else "norm < c√n"
    return {
        "metric_name": "noncommutative_L_infinity_norm",
        "metric_value": norm,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample_desc = "norm < c√n"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

infinity_norm', 'metric_value': 5.991830142481126, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 5.96133670580432, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.6215302279845885, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.14678256210607, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.0720069154488945, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.0618401665591835, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.15979419548317, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 5.889946751344519, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.579239868327553, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.456552365308594, 'instances_tested': 1, 'conjecture_holds': True,
TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 5.769737134003232, 'instances_tested': 1, 'conjecture_holds': True,

```

TRIAL: {'metric_name': 'noncommutative_L_infinity_norm', 'metric_value': 6.215313582659221, 'instance': 'ollama_remote'}
RESULT: SUPPORTED mean=6.141783163461413 std=0.2806895342283576 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111434
2	propose	ollama_remote	qwen3:8b	0	0	68614
3	preregistration	ollama_remote	qwen3:8b	0	0	24985
4	novelty	ollama_remote	qwen3:8b	0	0	20640
5	novelty	ollama_remote	qwen3:8b	0	0	13801
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14409
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9607
8	critic	ollama_remote	qwen3:8b	0	0	32402
9	judge	ollama_remote	qwen3:8b	0	0	21066

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 316959 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a956dba16ef5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a956dba16ef5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a956dba16ef5.tar.gz` (if generated)